



OPACUS: THE BEGINNER'S CHEAT SHEET

Differential Privacy (DP) in PyTorch: Opacus modifies standard PyTorch training to ensure that a model learns general population trends without memorizing the specifics of any individual data point. It achieves this by transforming standard SGD into DP-SGD.

1. The Core Concept: DP-SGD

Standard Stochastic Gradient Descent (SGD) computes gradients across a whole batch. Differentially Private SGD (DP-SGD) isolates them mathematically.

Standard SGD

- Computes batch-averaged gradients.
- Updates weights directly from the average.
- No privacy guarantees (vulnerable to inference).

DP-SGD (Opacus)

- Computes **per-sample** gradients independently.
- Clips** individual gradients to a maximum norm.
- Adds **Gaussian noise** to the aggregated gradients before updating.

2. The PrivacyEngine

The **PrivacyEngine** is the operational heart of Opacus. It wraps your standard PyTorch objects (Model, Optimizer, DataLoader) to automatically handle gradient isolation, clipping, noising, and accounting under the hood.

```
1 from opacus import PrivacyEngine
2
3 # 1. Initialize the engine
4 privacy_engine = PrivacyEngine()
5
6 # 2. Wrap standard PyTorch objects
7 model_dp, optimizer_dp, dataloader_dp = privacy_engine.make_private(
8     module=model,
9     optimizer=optimizer,
10    data_loader=dataloader,
11    noise_multiplier=1.2,
12    max_grad_norm=1.0,
13 )
```

Essential Hyperparameters

- noise_multiplier:** Determines the scale of the artificial noise injected into the gradients. Higher multiplier = tighter privacy but lower model accuracy.
- max_grad_norm:** The clipping threshold. It prevents any single data sample (like an outlier) from pulling the gradient too far in one direction, bounding the maximum influence of one individual.

3. Model Compatibility & Refactoring

Not all PyTorch layers are DP-compliant. **BatchNorm** is the primary offender. Because it normalizes data by mixing values across an entire batch, it violates the per-sample isolation required for DP-SGD.

Opacus provides **ModuleValidator** to audit and automatically rewrite architectures, swapping incompatible layers for safe ones (like GroupNorm).

```
1 from opacus.validators import ModuleValidator
2
3 # 1. Check if an off-the-shelf model is compliant
4 is_valid = ModuleValidator.is_valid(model)
5 # 2. Find exact architectural violations
6 errors = ModuleValidator.validate(model, strict=False)
7 # 3. Automatically rewrite the model (e.g., BatchNorm -> GroupNorm)
8 model_fixed = ModuleValidator.fix(model)
```

4. Tracking the Privacy Budget

Privacy leakage is quantified using two metrics that the **PrivacyEngine** tracks throughout training:

- ϵ (Epsilon):** The active privacy budget. It strictly increases every time the model views data. A lower final ϵ means a more private model.
- δ (Delta):** The probability of the privacy guarantee failing. This should always be set to a value strictly less than $1/N$, where N is your total dataset size.

```
1 # Delta should be < 1 / dataset_size (e.g., for 10,000 samples, use 1e-5)
2 target_delta = 1e-5
3
4 # Query the internal accountant for spent budget
5 epsilon = privacy_engine.get_epsilon(delta=target_delta)
6 print(f"Spent Privacy Budget:  $\epsilon$  = {epsilon:.2f}")
```

5. The Training Loop Rule

Once your objects are wrapped by the **PrivacyEngine**, your training loop remains functionally identical to standard PyTorch.

The complex mathematics happen invisibly during your standard calls:

- loss.backward():** Now computes per-sample gradients instead of a single batched gradient.
- optimizer_dp.step():** Now automatically clips those per-sample gradients, adds the *Gaussian noise*, updates the model weights, and logs the step in the privacy accountant.